

Estructuras de Datos

Clase 3 – Implementación de Pilas y Colas con Arreglos



Dr. Sergio A. Gómez
<http://cs.uns.edu.ar/~sag>



Departamento de Ciencias e Ingeniería de la Computación
 Universidad Nacional del Sur
 Bahía Blanca, Argentina

Cómo crear nuestras excepciones

- Las excepciones son subclases de la clase Exception de Java.
- Los métodos deben especificar en su signature/encabezado las excepciones que pueden lanzar/propagar:
`tipo método( parámetros formales ) throws clase_excepción`
- Una excepción se lanza desde una sentencia throw:
`throw new clase_excepción( parámetros actuales );`
- Las excepciones no manejadas en método, se propagan al método llamante.
- Ejemplo: Programar una clase persona. Una persona tiene un nombre y una edad. La edad debe ser un número positivo.
- Crearemos una excepción *PersonaException* para lanzar cuando el cliente ingresa una edad negativa en el constructor.

Estructuras de Datos - Dr. Sergio A. Gómez

2

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2026.

```
// PersonaException.java
public class PersonaException extends Exception {
    public PersonaException( String msg ) {
        super( msg );
    }
}

// Persona.java
public class Persona {
    private String nombre;
    private int edad;

    public Persona( String nombre, int edad ) throws PersonaException {
        this.nombre = nombre;
        if( edad <= 0 )
            throw new PersonaException( "Edad negativa: " + edad );
        this.edad = edad;
    }

    public String getNombre() { return nombre; }
    public int getEdad() { return edad; }
    public String toString() { return getNombre() + " " + getEdad(); }
}
```

```
// EjException03.java
public class EjException03
{
    public static void main( String [] args )
    {
        try {
            Persona p = new Persona( "Juan", 30 );
            System.out.println( "p: " + p );
            Persona q = new Persona( "Tito", p.getEdad() - 35 );
            System.out.println( "q: " + q );
        } catch( PersonaException e ) {
            System.out.println( e.getMessage() );
        }
    }
}
```

El constructor lanza excepción PersonaException porque la edad que recibe es negativa. El flujo de control salta al catch.

Estructuras de Datos - Dr. Sergio A. Gómez

4

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente: "Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2026.

Repaso: Tipo de dato abstracto

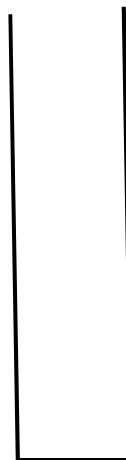
- Un *tipo de dato abstracto* (TDA) (en inglés, ADT por Abstract Data Type) es un tipo definido solamente en términos de sus operaciones y de las restricciones que valen entre las operaciones.
- Las restricciones las daremos en términos de comentarios.
- Cada TDA se representa en esta materia con una (o varias) interfaces.
- Una o más implementaciones del TDA se brindan en términos de clases concretas.
- Hoy implementaremos los TDAs Pila (Stack) y Cola (Queue) usando estructuras semiestáticas (arreglos).

Estructuras de datos - Dr. Sergio A. Gómez

5

Tipo de dato abstracto Pila

Pila: Colección lineal de objetos actualizada en un extremo llamado *tope* usando una política LIFO (last-in first-out, el primero en entrar es el último en salir).



Esta es una pila vacía

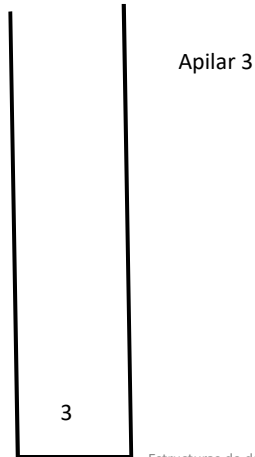
Estructuras de datos - Dr. Sergio A. Gómez

6

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente: "Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2026.

Tipo de dato abstracto Pila

Pila: Colección lineal de objetos actualizada en un extremo llamado *tope* usando una política LIFO (last-in first-out, el primero en entrar es el último en salir).

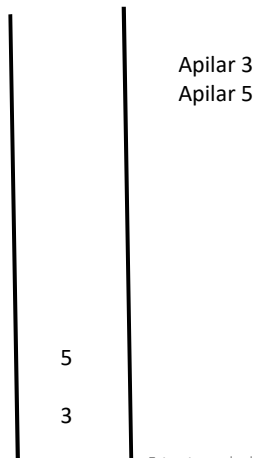


Estructuras de datos - Dr. Sergio A. Gómez

7

Tipo de dato abstracto Pila

Pila: Colección lineal de objetos actualizada en un extremo llamado *tope* usando una política LIFO (last-in first-out, el primero en entrar es el último en salir).



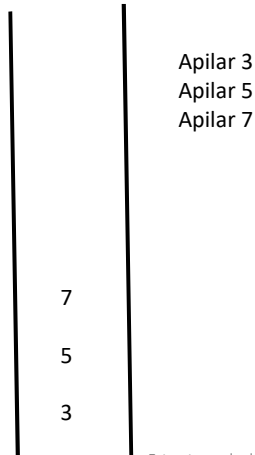
Estructuras de datos - Dr. Sergio A. Gómez

8

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
"Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2026.

Tipo de dato abstracto Pila

Pila: Colección lineal de objetos actualizada en un extremo llamado *tope* usando una política LIFO (last-in first-out, el primero en entrar es el último en salir).

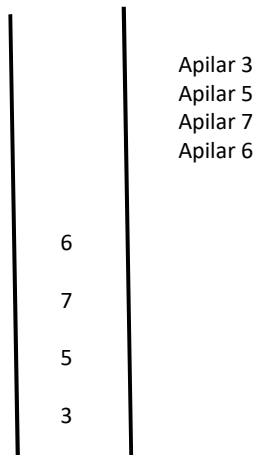


Estructuras de datos - Dr. Sergio A. Gómez

9

Tipo de dato abstracto Pila

Pila: Colección lineal de objetos actualizada en un extremo llamado *tope* usando una política LIFO (last-in first-out, el primero en entrar es el último en salir).



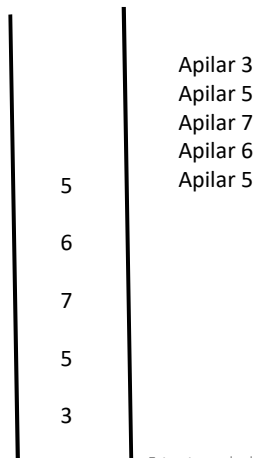
Estructuras de datos - Dr. Sergio A. Gómez

10

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2026.

Tipo de dato abstracto Pila

Pila: Colección lineal de objetos actualizada en un extremo llamado *tope* usando una política LIFO (last-in first-out, el primero en entrar es el último en salir).

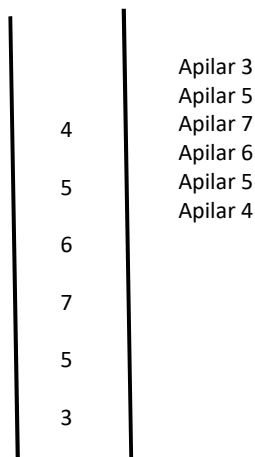


Estructuras de datos - Dr. Sergio A. Gómez

11

Tipo de dato abstracto Pila

Pila: Colección lineal de objetos actualizada en un extremo llamado *tope* usando una política LIFO (last-in first-out, el primero en entrar es el último en salir).



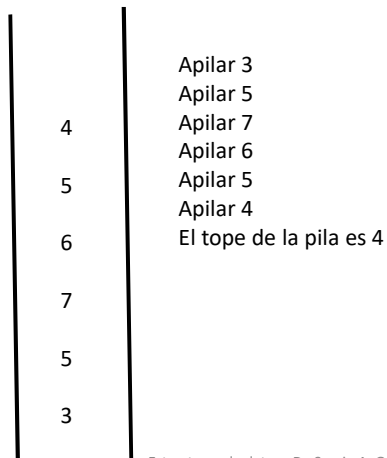
Estructuras de datos - Dr. Sergio A. Gómez

12

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2026.

Tipo de dato abstracto Pila

Pila: Colección lineal de objetos actualizada en un extremo llamado *tope* usando una política LIFO (last-in first-out, el primero en entrar es el último en salir).

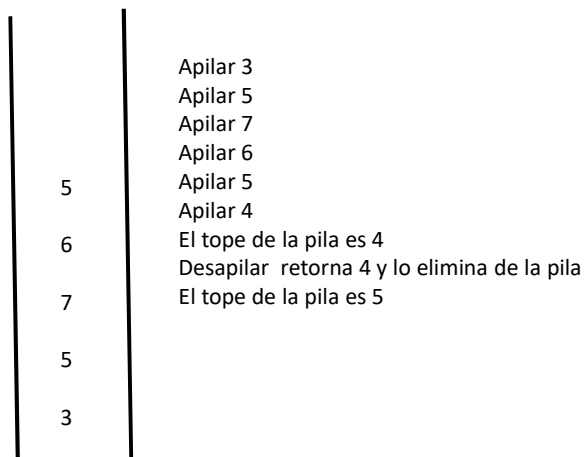


Estructuras de datos - Dr. Sergio A. Gómez

13

Tipo de dato abstracto Pila

Pila: Colección lineal de objetos actualizada en un extremo llamado *tope* usando una política LIFO (last-in first-out, el primero en entrar es el último en salir).



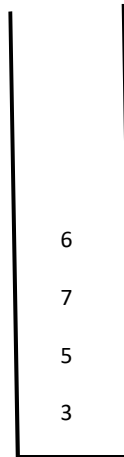
Estructuras de datos - Dr. Sergio A. Gómez

14

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2026.

Tipo de dato abstracto Pila

Pila: Colección lineal de objetos actualizada en un extremo llamado *tope* usando una política LIFO (last-in first-out, el primero en entrar es el último en salir).



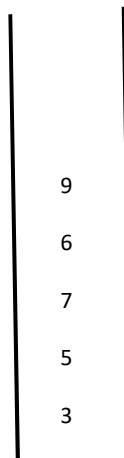
Apilar 3
 Apilar 5
 Apilar 7
 Apilar 6
 Apilar 5
 Apilar 4
 El tope de la pila es 4
 Desapilar retorna 4 y lo elimina de la pila
 El tope de la pila es 5
 Desapilar retorna 5 y lo elimina de la pila

Estructuras de datos - Dr. Sergio A. Gómez

15

Tipo de dato abstracto Pila

Pila: Colección lineal de objetos actualizada en un extremo llamado *tope* usando una política LIFO (last-in first-out, el primero en entrar es el último en salir).



Apilar 3
 Apilar 5
 Apilar 7
 Apilar 6
 Apilar 5
 Apilar 4
 El tope de la pila es 4
 Desapilar retorna 4 y lo elimina de la pila
 El tope de la pila es 5
 Desapilar retorna 5 y lo elimina de la pila
 Apilar 9
 El tope de la pila es 9

Estructuras de datos - Dr. Sergio A. Gómez

16

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 "Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2026.

TDA Pila (Stack) de acuerdo a GT

Operaciones:

- **push(e):** Inserta el elemento e en el tope de la pila.
- **pop():** Elimina el elemento del tope de la pila y lo entrega como resultado. Si se aplica a una pila vacía, lanza una excepción `EmptyStackException`.
- **isEmpty():** Retorna verdadero si la pila no contiene elementos y falso en caso contrario.
- **top():** Retorna el elemento del tope de la pila. Si se aplica a una pila vacía, lanza una excepción `EmptyStackException`.
- **size():** Retorna un entero positivo que indica cuántos elementos hay en la pila.

Estructuras de datos - Dr. Sergio A. Gómez

17

Implementación de Pila

Definición de una interfaz Pila:

- Se abstrae de la ED con la que se implementará
- Se documenta el significado de cada método en lenguaje natural
- Se usa un parámetro formal de tipo representando el tipo de los elementos de la pila
- Se definen excepciones para las condiciones de error

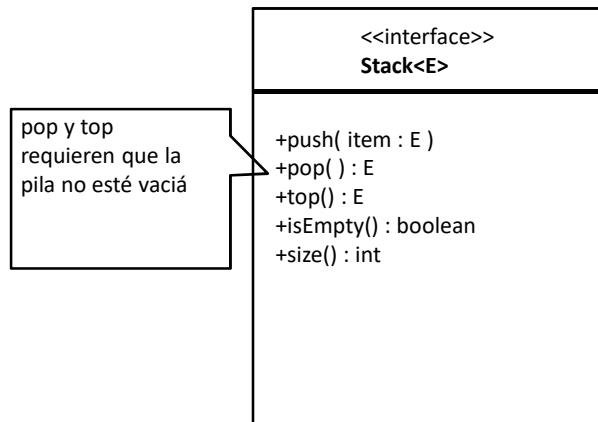
Estructuras de datos - Dr. Sergio A. Gómez

18

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente: *"Estructuras de Datos. Notas de Clase"*. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2026.

Interfaz Stack en UML

En UML las operaciones se dan con una sintaxis Pascal-like y las excepciones se dan como comentarios:



Estructuras de datos - Dr. Sergio A. Gómez

19

Código Java en archivo *Stack.java*: Los comentarios explican semántica y precondiciones.

```

public interface Stack<E> {
    // Inserta item en el tope de la pila.
    public void push( E item );

    // Retorna true si la pila está vacía y falso en caso contrario.
    public boolean isEmpty();

    // Elimina el elemento del tope de la pila y lo retorna.
    // Lanza una excepción EmptyStackException si la pila está vacía.
    public E pop();

    // Retorna el elemento del tope de la pila y lo retorna.
    // Lanza una excepción EmptyStackException si la pila está vacía..
    public E top();

    // Retorna la cantidad de elementos de la pila.
    public int size();
}

```

Estructuras de datos - Dr. Sergio A. Gómez

20

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2026.

Nota: Los comentarios a la hora de programar los haremos como comentarios Javadoc :

```
/**
 * Inserta un elemento en el tope de la pila.
 * @param element Elemento a insertar.
 */
public void push(E element);

/**
 * Remueve el elemento que se encuentra en el tope de la pila.
 * @return Elemento removido.
 * @throws EmptyStackException si la pila está vacía.
 */
public E pop() throws EmptyStackException;
```

Nota: Luego se compilan con la herramienta Javadoc.exe para generar un sitio web que forma la documentación del código fuente y se ve así:

Method Summary

All Methods	Instance Methods	Abstract Methods
boolean	isEmpty()	Consulta si la pila está vacía.
E	pop()	Remueve el elemento que se encuentra en el tope de la pila.
void	push(E element)	Inserta un elemento en el tope de la pila.
int	size()	Consulta la cantidad de elementos de la pila.
E	top()	Examina el elemento que se encuentra en el tope de la pila.

push

void push(E element)

Inserta un elemento en el tope de la pila.

Parameters:
element - Elemento a insertar.

pop

E pop()

throws EmptyStackException

Remueve el elemento que se encuentra en el tope de la pila.

Returns:
Elemento removido.

Throws:
EmptyStackException - si la pila está vacía.

Nota: Es importante escribir los comentarios Javadoc junto con el código y no al final porque los mismos se usan en el Intellisense (i.e. los globitos amarillos de ayuda del autocompletado de código en el editor de código de Eclipse).

```
public class App {
    public static void main(String[] args) {
        Stack<Integer> s;
    }
}
```

- hashCode() : int - Object
- isEmpty() : boolean - Stack
- notify() : void - Object
- notifyAll() : void - Object
- pop() : Integer - Stack
- push(Integer element) : void - Stack
- size() : int - Stack
- top() : Integer - Stack
- toString() : String - Object
- wait() : void - Object
- wait(long arg0) : void - Object
- wait(long arg0, int arg1) : void - Object

Inserta un elemento en el tope de la pila.
Parameters:
element Elemento a insertar.

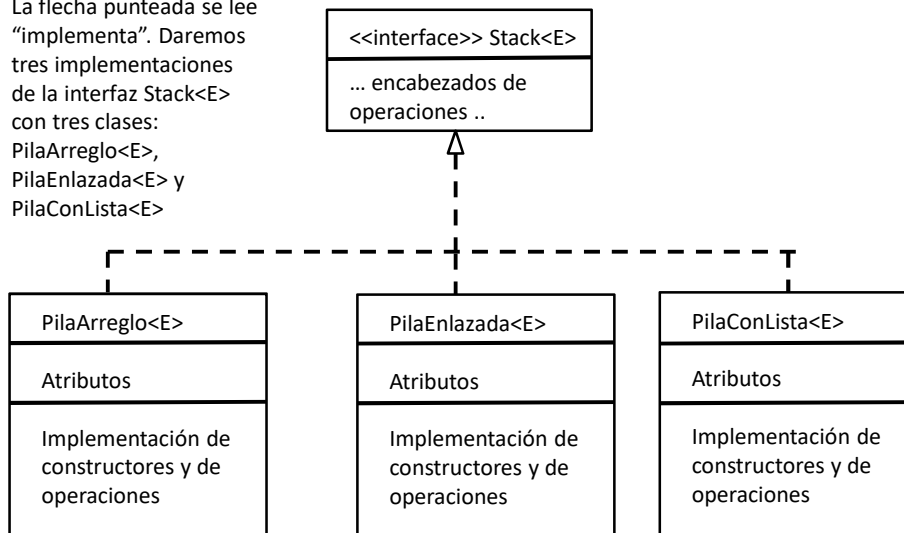
Implementaciones tradicionales de pilas

- 1) Con un arreglo
- 2) Con una estructura de nodos enlazados
- 3) En términos de una lista (lo dejamos pendiente hasta dar el TDA Lista)

23

Diagrama UML del diseño

La flecha punteada se lee "implementa". Daremos tres implementaciones de la interfaz Stack<E> con tres clases: PilaArreglo<E>, PilaEnlazada<E> y PilaConLista<E>



Estructuras de datos - Dr. Sergio A. Gómez

24

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente: "Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2026.

Excepciones para la pila

- Usaremos subclases de RuntimeException.
- Esto permite escribir código usando try-catch para capturar la excepción o propagarla si no lo usamos.
- En un archivo EmptyStackException.java:

```
public class EmptyStackException extends RuntimeException {
    public EmptyStackException(String mensajeDeError) {
        super(mensajeDeError);
    }
}
```

Recuerde que heredamos los métodos:

```
public String getMessage(); // Recupera el mensaje de error
public void printStackTrace(); // Imprime pila de registros de
activación con el número de línea que falló en cada método.
```

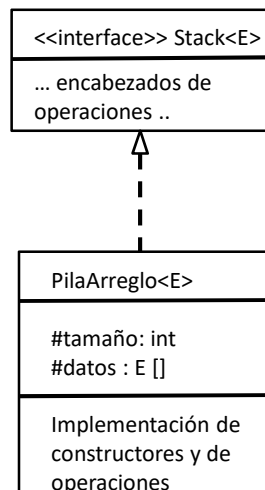
Estructuras de datos - Dr. Sergio A. Gómez

25

Implementación de pila con arreglo

Estructura de datos para mantener el estado de los elementos de la pila:

- Un entero positivo *tamaño* que indica la dimensión del arreglo.
- Un arreglo *datos* que contiene tamaño elementos apilados.
- El elemento en la posición *tamaño* - 1 es el tope de la pila.
- El elemento en la posición 0 es el fondo de la pila.



Estructuras de datos - Dr. Sergio A. Gómez

26

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente: "Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2026.

Mostramos la clase *PilaArreglo* (para completar) y otra clase *Aplicación...* que la utiliza para armar la pila s:

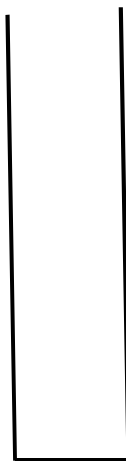
```
// Archivo: PilaArreglo.java
public class PilaArreglo<E> implements Stack<E> {
    ...
}
```

```
// Archivo: AplicacionUsaPilaArreglo.java
public class AplicacionUsaPilaArreglo {
    public static void main( String [] args ) {

    }
}
```

27

Mostramos la clase *PilaArreglo* (para completar) y otra clase *Aplicación...* que la utiliza para armar la pila s:



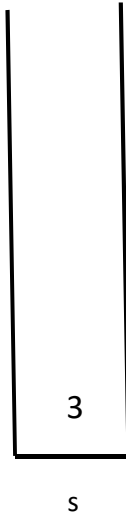
```
// Archivo: PilaArreglo.java
public class PilaArreglo<E> implements Stack<E> {
    ...
}
```

```
// Archivo: AplicacionUsaPilaArreglo.java
public class AplicacionUsaPilaArreglo {
    public static void main( String [] args ) {
        Stack<Integer> s = new PilaArreglo<Integer>();

    }
}
```

28

Mostramos la clase *PilaArreglo* (para completar) y otra clase *Aplicación...* que la utiliza para armar la pila s:

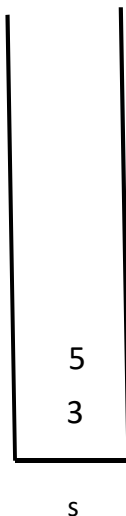


```
// Archivo: PilaArreglo.java
public class PilaArreglo<E> implements Stack<E> {
    ...
}
```

```
// Archivo: AplicacionUsaPilaArreglo.java
public class AplicacionUsaPilaArreglo {
    public static void main( String [] args ) {
        Stack<Integer> s = new PilaArreglo<Integer>();
        s.push( 3 );
    }
}
```

29

Mostramos la clase *PilaArreglo* (para completar) y otra clase *Aplicación...* que la utiliza para armar la pila s:

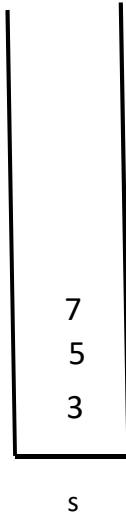


```
// Archivo: PilaArreglo.java
public class PilaArreglo<E> implements Stack<E> {
    ...
}
```

```
// Archivo: AplicacionUsaPilaArreglo.java
public class AplicacionUsaPilaArreglo {
    public static void main( String [] args ) {
        Stack<Integer> s = new PilaArreglo<Integer>();
        s.push( 3 );
        s.push( 5 );
    }
}
```

30

Mostramos la clase *PilaArreglo* (para completar) y otra clase *Aplicación...* que la utiliza para armar la pila s:

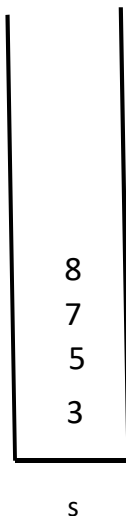


```
// Archivo: PilaArreglo.java
public class PilaArreglo<E> implements Stack<E> {
    ...
}
```

```
// Archivo: AplicacionUsaPilaArreglo.java
public class AplicacionUsaPilaArreglo {
    public static void main( String [] args ) {
        Stack<Integer> s = new PilaArreglo<Integer>();
        s.push( 3 );
        s.push( 5 );
        s.push( 7 );
    }
}
```

31

Mostramos la clase *PilaArreglo* (para completar) y otra clase *Aplicación...* que la utiliza para armar la pila s:

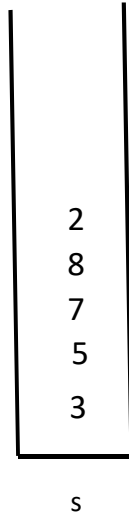


```
// Archivo: PilaArreglo.java
public class PilaArreglo<E> implements Stack<E> {
    ...
}
```

```
// Archivo: AplicacionUsaPilaArreglo.java
public class AplicacionUsaPilaArreglo {
    public static void main( String [] args ) {
        Stack<Integer> s = new PilaArreglo<Integer>();
        s.push( 3 );
        s.push( 5 );
        s.push( 7 );
        s.push( 8 );
    }
}
```

32

Mostramos la clase *PilaArreglo* (para completar) y otra clase *Aplicación...* que la utiliza para armar la pila s:

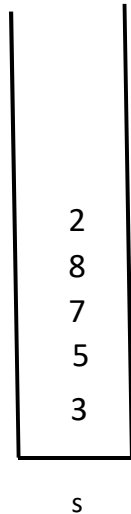


```
// Archivo: PilaArreglo.java
public class PilaArreglo<E> implements Stack<E> {
    ...
}
```

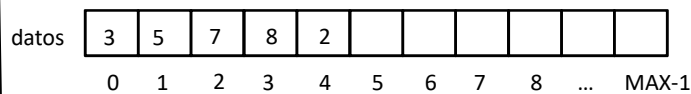
```
// Archivo: AplicacionUsaPilaArreglo.java
public class AplicacionUsaPilaArreglo {
    public static void main( String [] args ) {
        Stack<Integer> s = new PilaArreglo<Integer>();
        s.push( 3 );
        s.push( 5 );
        s.push( 7 );
        s.push( 8 );
        s.push( 2 );
    }
}
```

33

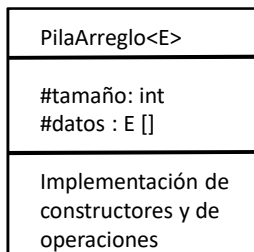
Pila: Atributos de la implementación con arreglo junto con su representación



```
// Archivo: PilaArreglo.java
public class PilaArreglo<E> implements Stack<E> {
    protected int tamaño;
    protected E [] datos;
    ...
}
```

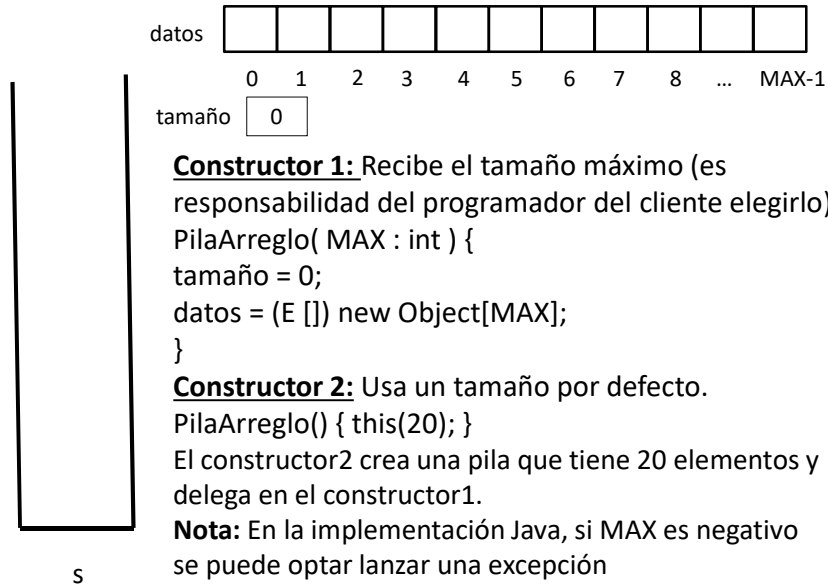


tamaño 5

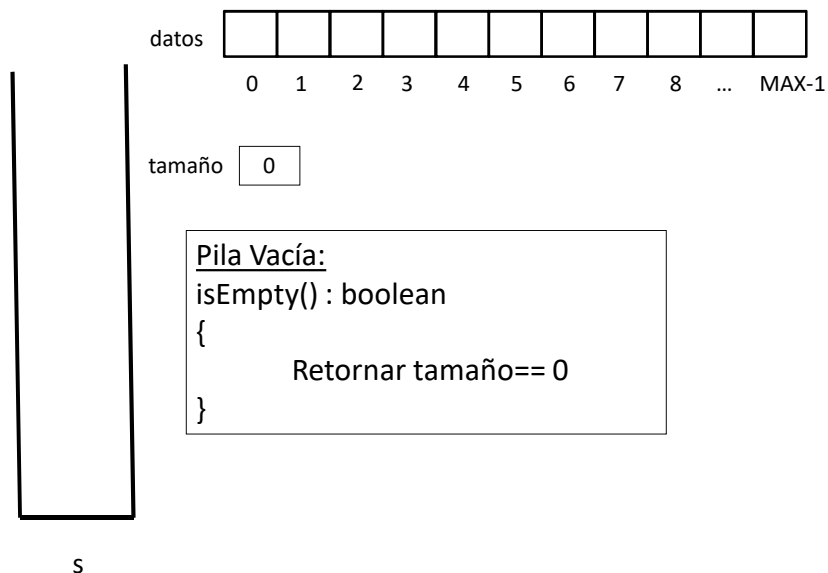


34

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente: "Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2026.

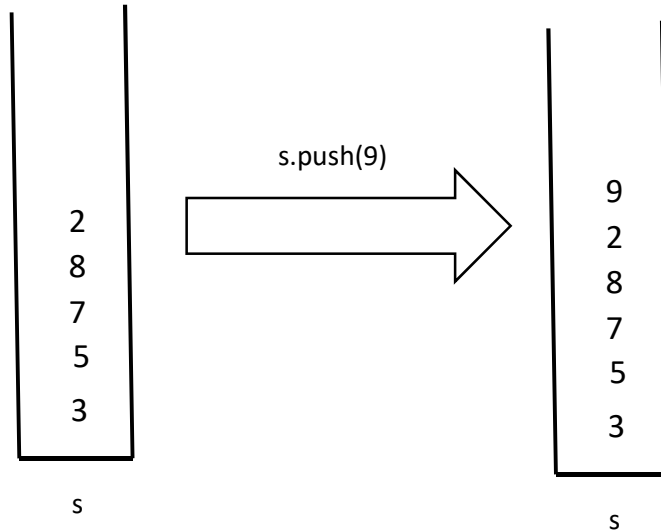
Constructor: Crea una pila vacía

35

Implementación de isEmpty

36

Implementación de Apilar: Dada la pila de la izquierda luego de apilar un 9 obtenemos la pila de la derecha



37

Implementación de Apilar

Opción1: Si hay lugar, insertar el nuevo elemento al final del arreglo. Si no hay lugar, generar una situación de error lanzando una excepción *FullStackException*.

Pseudocódigo:

```

Apilar(item : E)
Si tamaño == TAMAÑO DEL ARREGLO
entonces
    error ("Pila llena")
Sino
    datos[tamaño] = item
    tamaño = tamaño + 1
  
```

Nota:

Soluciones al problema de pila llena:

- 1) Utilizar una excepción *FullStackException* para modelar el error de que la pila se llenó (como se mostró acá).
- 2) Cuando la pila se llena, crear un arreglo de datos más grande (para resolver en la práctica y es la opción que adoptaremos para dar más robustez y flexibilidad al cliente).

Implementación en Java:

```

public void push(E item) {
    if (tamaño == datos.length)
        throw new FullStackException("Pila llena");
    datos[tamaño++] = item;
}
  
```

Nota: En la tradición de C/C++/Java/JavaScript, muchas veces no se pone el else adhiriendo al patrón de refactoring de *guard clause* (ver diapositiva siguiente).

38

Patrón de refactoring de guard clause

El patrón de cláusula de guarda es una técnica de programación que utiliza sentencias condicionales al inicio de una función o método para comprobar si hay entradas no válidas o casos límite, y salir inmediatamente (normalmente con una sentencia `return` anticipada o lanzando una excepción) si no se cumplen dichas condiciones. Este enfoque se utiliza para evitar el anidamiento profundo de sentencias `if-else`, lo que da como resultado un código más plano, legible y fácil de mantener.

```
public double getPayAmount() {
    double result;
    if (isDead){
        result = deadAmount();
    }
    else {
        if (isSeparated){
            result = separatedAmount();
        }
        else {
            if (isRetired){
                result = retiredAmount();
            }
            else{
                result = normalPayAmount();
            }
        }
    }
    return result;
}
```

```
public double getPayAmount() {
    if (isDead){
        return deadAmount();
    }
    if (isSeparated){
        return separatedAmount();
    }
    if (isRetired){
        return retiredAmount();
    }
    return normalPayAmount();
}
```

39

Discusión sobre apilar con pila llena

El cliente de la pila debe determinar el tamaño con la cual crearla para evitar que se llene.

Cuando la pila se llena, hay 2 estrategias para resolverlo:

1. Lanzar una excepción indicando que la estructura está llena (FullStackException).
2. Aumentar el tamaño de la colección subyacente de datos (crear un nuevo arreglo, copiar los datos del arreglo original de datos en el nuevo, hacer que el arreglo nuevo sea el arreglo de datos, el recolector de basura se lleva el arreglo viejo)

La opción 1 es más fácil de programar pero le genera problemas al cliente cuando debe decidir el tamaño inicial y cuando la estructura se llena.

La opción 2 es más difícil de programar pero le da más flexibilidad al cliente para crear la estructura y la estructura tiene un tamaño no acotado.

La opción 1 no tiene sobrecarga en ejecución porque siempre tarda lo mismo.

La opción 2 tiene sobrecarga en ejecución cuando se llena porque debe generar un nuevo arreglo y copiar los datos en él.

40

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente: "Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2026.

Implementación de apilar

Opción2: Cuando el arreglo esté lleno, incrementar el tamaño del arreglo (por ejemplo agregarle 10 elementos), luego copiar los elementos al nuevo arreglo y después insertar el nuevo elemento.

Ventaja: La pila puede ahora crecer en forma no acotada.

Desventaja: Crear un arreglo nuevo más grande, requiere copiar todos los elementos del arreglo viejo en el nuevo (ineficiente).

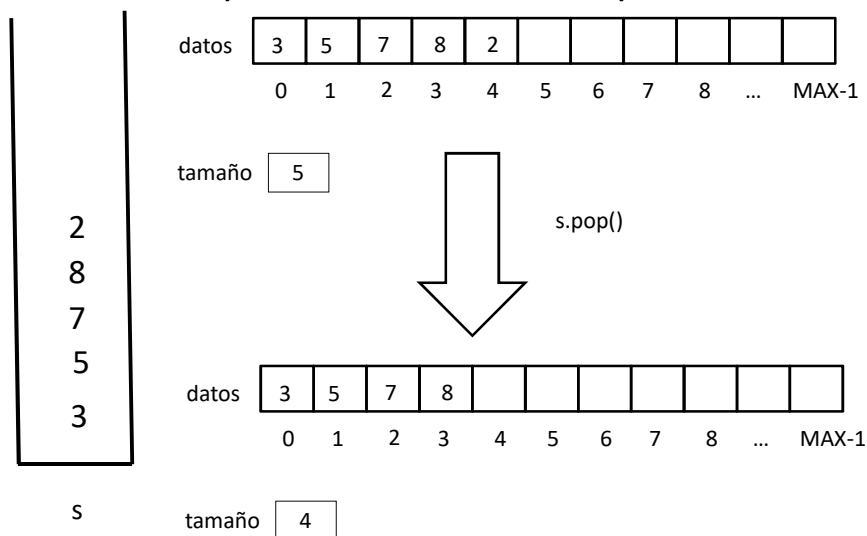
Nota: Le podemos dar libertad al usuario con un nuevo constructor que reciba además del tamaño inicial, el incremento a utilizar cada vez que rebalsa el arreglo

PilaConArreglo(maximoTamaño : int; incrementoTamaño : int);

Estructuras de datos- Dr. Sergio A. Gómez

41

Implementación de desapilar



La ejecución de la operación `pop()` elimina el 2 del tope de la pila `s`.

42

Implementación de desapilar

2
8
7
5
3

s

datos

3	5	7	8	2					
---	---	---	---	---	--	--	--	--	--

0 1 2 3 4 5 6 7 8 ... MAX-1

tamaño 5

top() : E
Si isEmpty() entonces
error ("Pila vacía")
Sino
retornar datos[tamaño-1]

pop() : E
resultado $\leftarrow$ top()
datos[tamaño-1] = null
tamaño = tamaño - 1
retornar resultado

Notas: El error se modela con excepción EmptyStackException.
Si falla top(), entonces falla pop().

43

TDA Cola

Frente
(head)
Rabo
(tail)

←

←

1 5 6 7 2 6 8 9 8

Dato saliente o
desencolado
Dato entrante o
encolado

Una *cola* (queue) es una colección con modelo de secuencia cuyos datos ingresan por un extremo llamado *rabo* (tail) y salen por otro extremo llamado *frente* (head).

Esta política de actualización se llama FIFO (por *first-in first-out*) o también FCFS (*first-come first-served*).

44

TDA Cola

Operaciones:

- enqueue(e): Inserta el elemento *e* en el rabo de la cola
- dequeue(): Elimina el elemento del frente de la cola y lo retorna. Si la cola está vacía, lanza excepción `EmptyQueueException`.
- front(): Retorna el elemento del frente de la cola. Si la cola está vacía, lanza excepción `EmptyQueueException`.
- isEmpty(): Retorna verdadero si la cola no tiene elementos y falso en caso contrario
- size(): Retorna la cantidad de elementos de la cola.

45

Implementación de Cola

Definición de una interfaz Cola:

- Se abstrae de la ED con la que se implementará
- Se documenta el significado de cada método en lenguaje natural
- Se usa un parámetro formal de tipo representando el tipo de los elementos de la cola
- Se definen excepciones para las condiciones de error

Estructuras de datos - Dr. Sergio A. Gómez

46

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente: "Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2026.

```

public interface Queue<E> {
    // Inserta el elemento e al final de la cola
    public void enqueue(E item);

    // Elimina el elemento del frente de la cola y lo retorna.
    // Si la cola está vacía, lanza excepción EmptyQueueException
    public E dequeue();

    // Retorna el elemento del frente de la cola.
    // Si la cola está vacía, lanza excepción EmptyQueueException
    public E front();

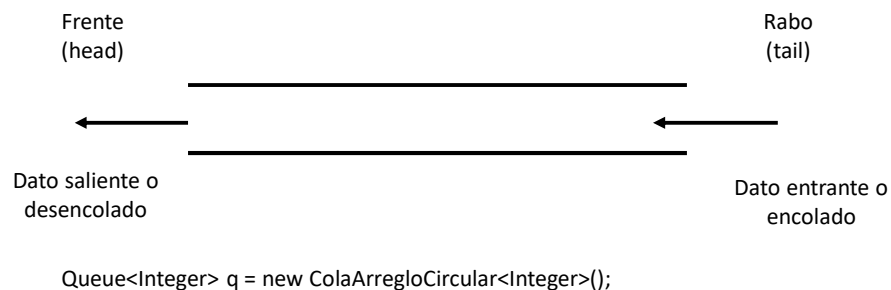
    // Retorna verdadero si la cola no tiene elementos
    // y falso en caso contrario
    public boolean isEmpty();

    // Retorna la cantidad de elementos de la cola.
    public int size();
}

```

47

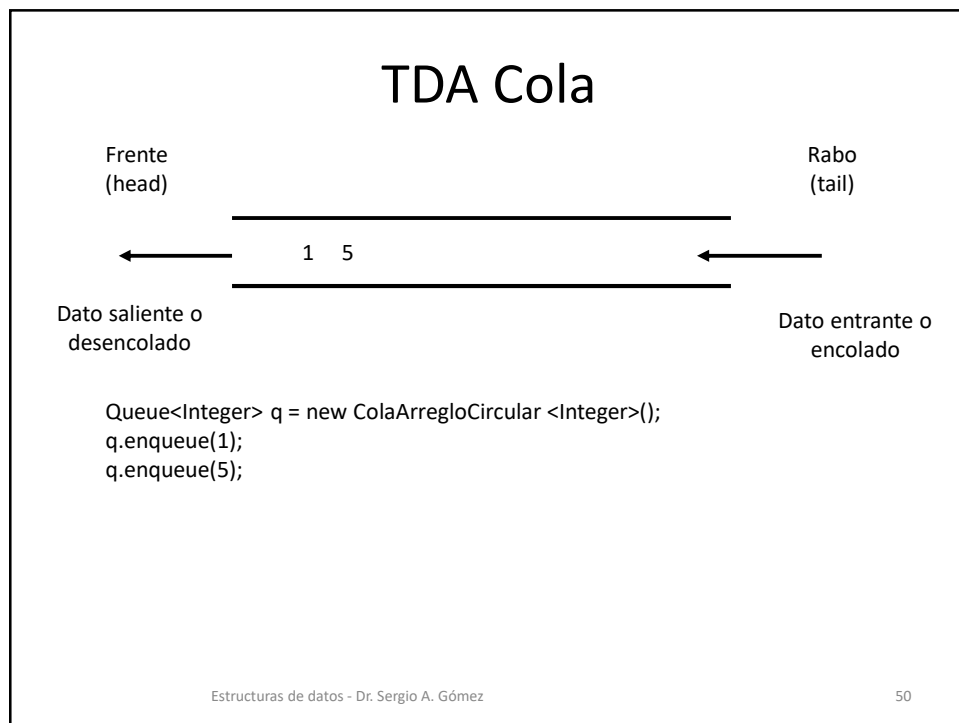
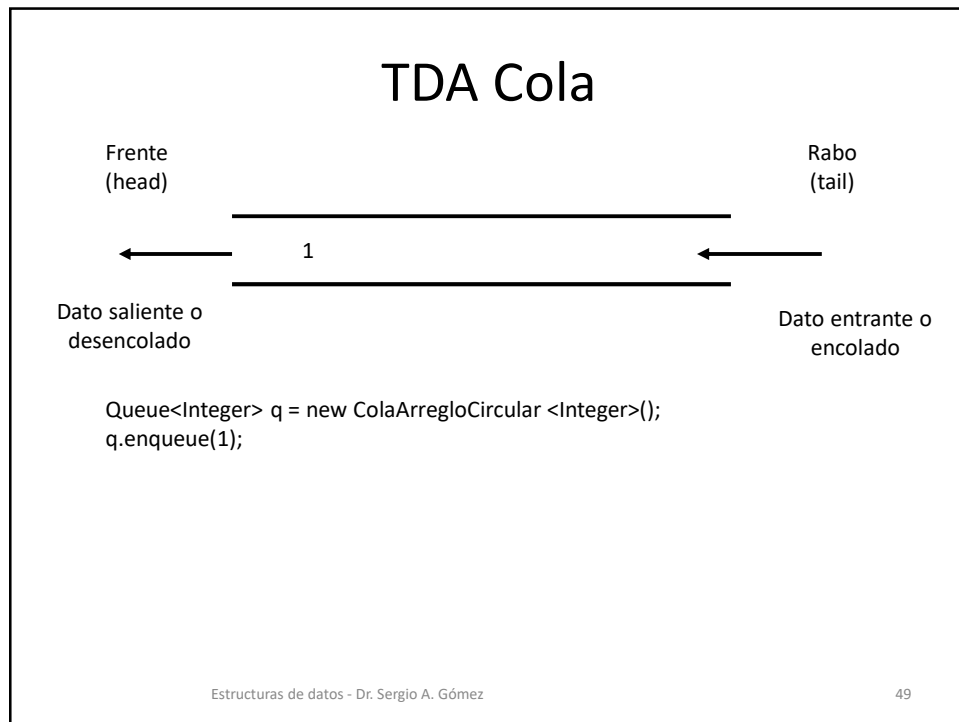
TDA Cola



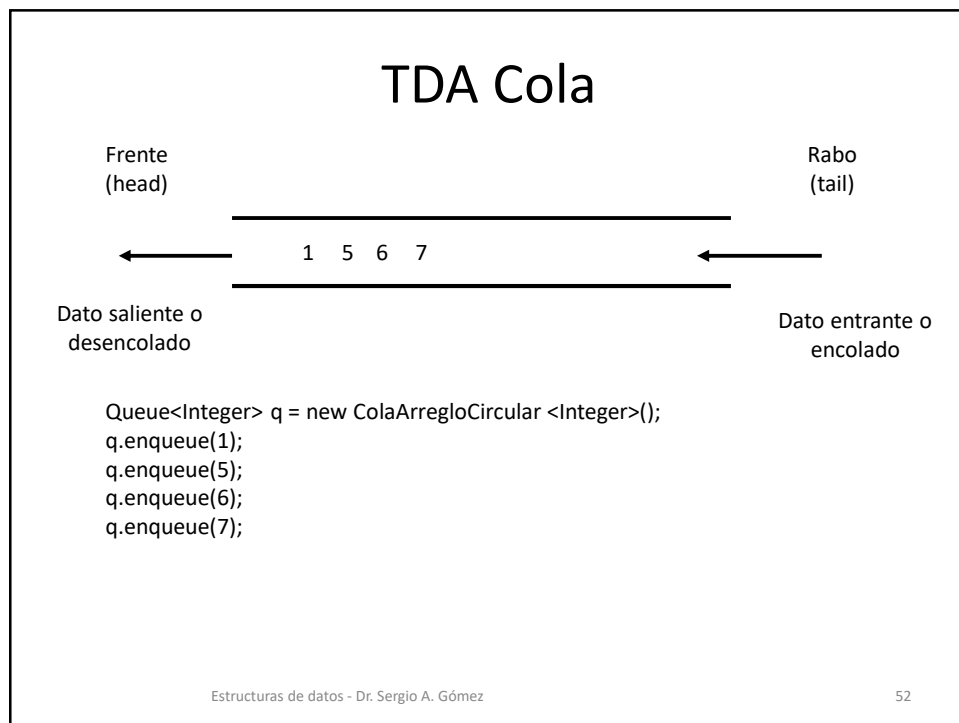
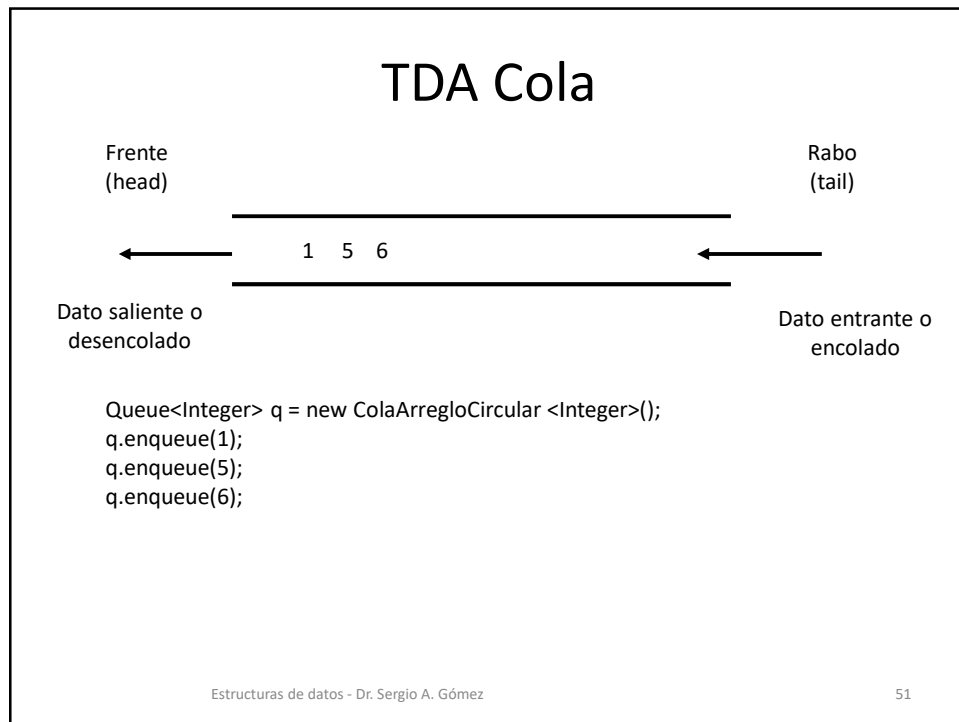
Estructuras de datos - Dr. Sergio A. Gómez

48

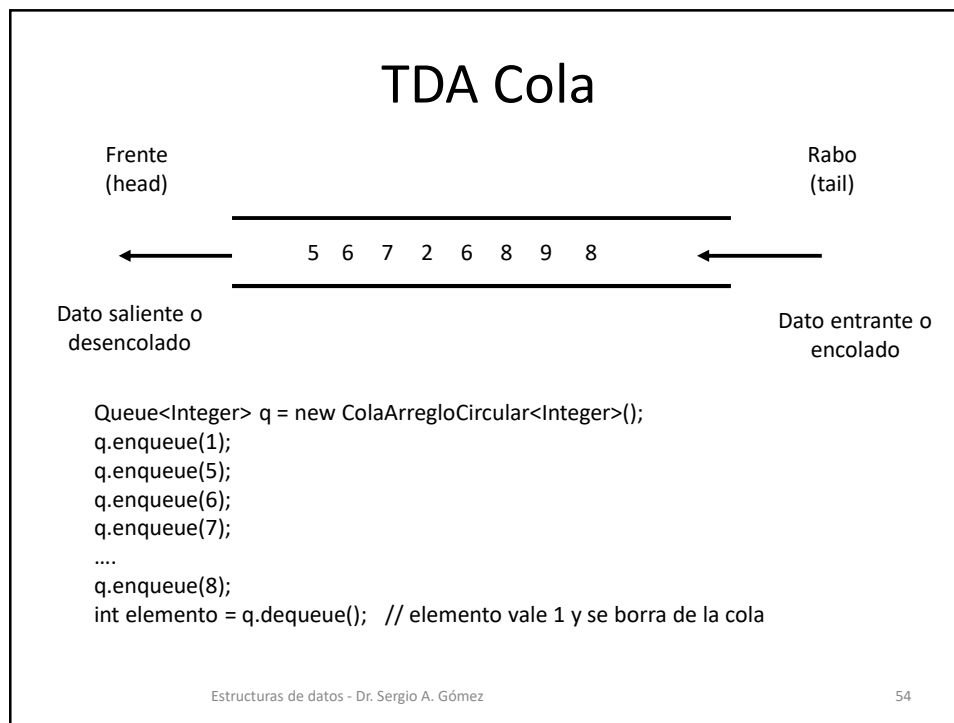
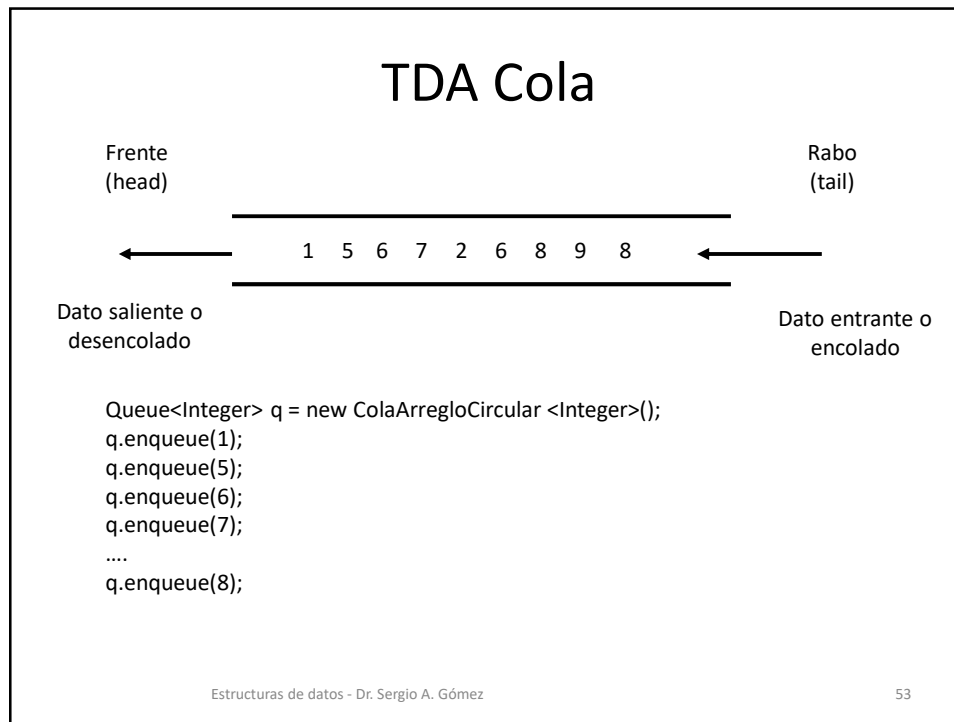
El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2026.



El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
"Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2026.



El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
"Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2026.



El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
"Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2026.

TDA Cola

Frente
(head)

←

6 7 2 6 8 9 8

→

Rabo
(tail)

Dato saliente o desencholado Dato entrante o encolado

```

Queue<Integer> q = new ColaArregloCircular<Integer>();
q.enqueue(1);
q.enqueue(5);
q.enqueue(6);
q.enqueue(7);
....
q.enqueue(8);
int elemento = q.dequeue(); // elemento vale 1 y se borra de la cola
q.dequeue();               // borro el 5 de la cola y no lo almaceno
  
```

Estructuras de datos - Dr. Sergio A. Gómez 55

TDA Cola

Frente
(head)

←

6 7 2 6 8 9 8 4

→

Rabo
(tail)

Dato saliente o desencholado Dato entrante o encolado

```

Queue<Integer> q = new ColaArregloCircular<Integer>();
q.enqueue(1);
q.enqueue(5);
q.enqueue(6);
q.enqueue(7);
....
q.enqueue(8);
int elemento = q.dequeue(); // elemento vale 1 y se borra de la cola
q.dequeue();               // borro el 2 de la cola y no lo almaceno
q.enqueue(4);              // Puedo encolar cuando quiera
  
```

Estructuras de datos - Dr. Sergio A. Gómez 56

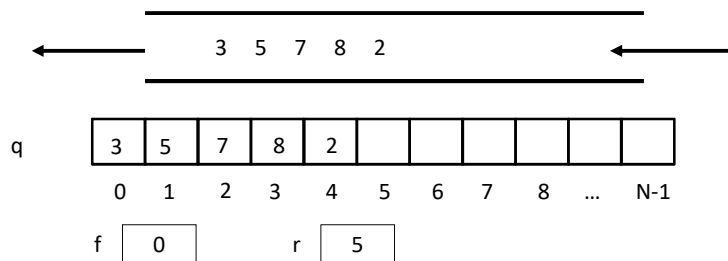
El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2026.

Implementaciones de colas

- 1) Con un arreglo circular
- 2) Con una estructura enlazada (más adelante)
- 3) En términos de una lista (lo dejamos pendiente hasta dar el TDA Lista)

57

Implementación de cola con un arreglo circular



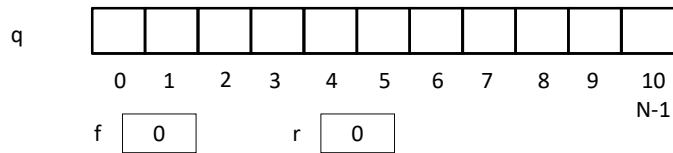
- q es un arreglo de N componentes y mantiene los elementos de la cola
- El tamaño máximo de q es $N-1$ (lo que permite diferenciar la cola vacía de la cola llena)
- f es la posición en q del próximo elemento a eliminar en un dequeue
- r es la posición en la cual se va a insertar el siguiente elemento con un enqueue.

Estructuras de datos - Dr. Sergio A. Gómez

58

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente: "Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2026.

Implementación de cola con un arreglo circular



Constructor:

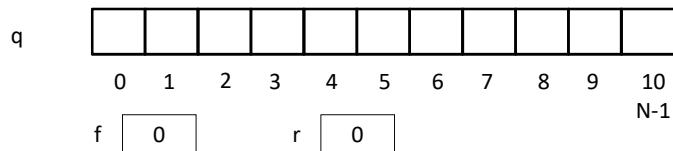
f = 0

r = 0

Estructuras de datos - Dr. Sergio A. Gómez

59

Implementación de cola con un arreglo circular



Constructor:

f = 0

r = 0

isEmpty():

Retornar f == r

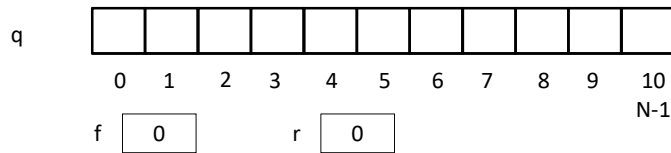
En este caso isEmpty retorna verdadero.

Estructuras de datos - Dr. Sergio A. Gómez

60

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2026.

Implementación de cola con un arreglo circular



Constructor:

$f = 0$
 $r = 0$

isEmpty():

Retornar $f == r$

front():

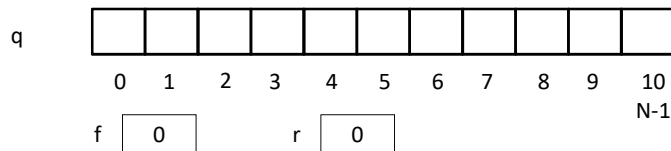
Si isEmpty() entonces
error("cola vacía")
Sino
retornar $q[f]$

En este caso, front() daría error.
Al implementarlo en Java debe usar
excepciones para señalar un error.

Estructuras de datos - Dr. Sergio A. Gómez

61

Implementación de cola con un arreglo circular



Size():

Retornar $(N-f+r) \bmod N$

enqueue(e):

Si size() == N-1 entonces
error("cola llena")
Sino

$q[r] = e$
 $r = (r+1) \bmod N$

Nota: La cola puede almacenar N-1
elementos a lo sumo.

Nota: En este caso, size retorna
 $(N-f+r) \% N = (11-0+0)\%11 = 11 \% 11 = 0$

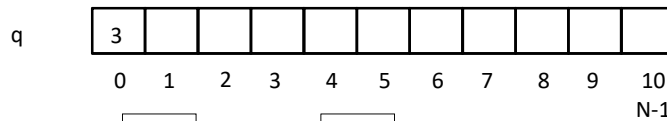
Nota: Otra opción más sencilla es usar un
atributo más para representar size.

Estructuras de datos - Dr. Sergio A. Gómez

62

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
"Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2026.

¿Cómo funciona encolar?: Encolo 3



Size():
Retornar $(N-f+r) \bmod N$

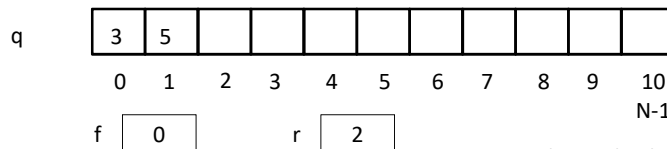
Nota: La cola puede almacenar N-1 elementos a lo sumo. r vale 1 porque apunta al siguiente slot vacío.

enqueue(e):
Si $size() == N-1$ entonces
 error(“cola llena”)
Sino
 $q[r] = e$
 $r = (r+1) \bmod N$

Estructuras de datos - Dr. Sergio A. Gómez

63

¿Cómo funciona encolar?: Encolo 5



Size():
Retornar $(N-f+r) \bmod N$

Nota: La cola puede almacenar N-1 elementos a lo sumo

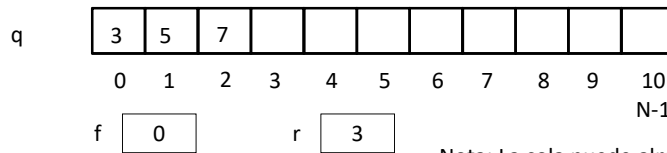
Nota: r apunta a 2 que es el siguiente slot vacío

enqueue(e):
Si $size() == N-1$ entonces
 error(“cola llena”)
Sino
 $q[r] = e$
 $r = (r+1) \bmod N$

Estructuras de datos - Dr. Sergio A. Gómez

64

¿Cómo funciona encolar?: Encolo 7



Size():
Retornar $(N-f+r) \bmod N$

Nota: La cola puede almacenar N-1 elementos a lo sumo

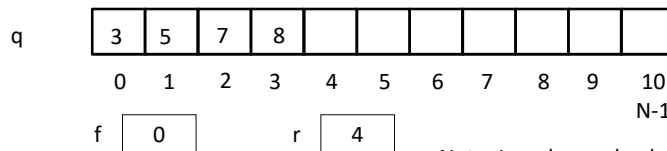
Nota: r apunta a 3 que el siguiente slot vacío.

enqueue(e):
Si $size() == N-1$ entonces
 error("cola llena")
Sino
 $q[r] = e$
 $r = (r+1) \bmod N$

Estructuras de datos - Dr. Sergio A. Gómez

65

¿Cómo funciona encolar?: Encolo 8



Size():
Retornar $(N-f+r) \bmod N$

Nota: La cola puede almacenar N-1 elementos a lo sumo

Nota: r apunta a 4 que es el siguiente slot vacío.

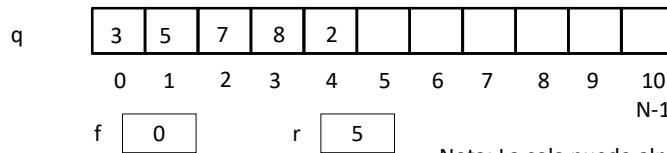
enqueue(e):
Si $size() == N-1$ entonces
 error("cola llena")
Sino
 $q[r] = e$
 $r = (r+1) \bmod N$

Estructuras de datos - Dr. Sergio A. Gómez

66

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
"Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2026.

¿Cómo funciona encolar?: Encolo 2



Size():
Retornar $(N-f+r) \bmod N$

Nota: La cola puede almacenar N-1 elementos a lo sumo

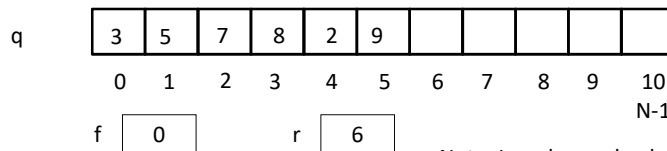
Nota: r apunta a 5 que es el siguiente slot vacío

enqueue(e):
Si $size() == N-1$ entonces
 error(“cola llena”)
Sino
 $q[r] = e$
 $r = (r+1) \bmod N$

Estructuras de datos - Dr. Sergio A. Gómez

67

¿Cómo funciona encolar?: Encolo 9



Size():
Retornar $(N-f+r) \bmod N$

Nota: La cola puede almacenar N-1 elementos a lo sumo

Nota: r apunta a 6 que es el siguiente slot vacío.

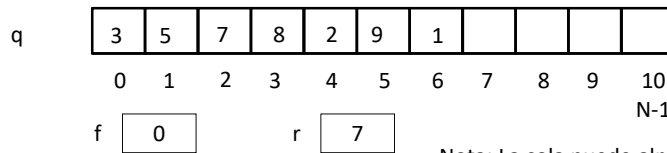
enqueue(e):
Si $size() == N-1$ entonces
 error(“cola llena”)
Sino
 $q[r] = e$
 $r = (r+1) \bmod N$

Estructuras de datos - Dr. Sergio A. Gómez

68

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
“Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2026.

¿Cómo funciona encolar?: Encolo 1



Size():

Retornar $(N-f+r) \bmod N$

Nota: La cola puede almacenar N-1 elementos a lo sumo

Nota: r apunta a 7 que es el siguiente slot vacío.

enqueue(e):

Si $size() == N-1$ entonces
error("cola llena")

Sino

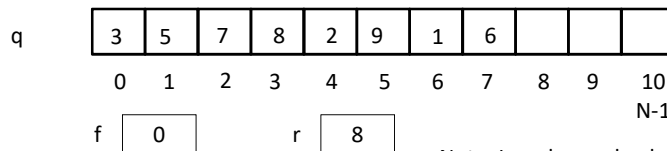
$q[r] = e$

$r = (r+1) \bmod N$

Estructuras de datos - Dr. Sergio A. Gómez

69

¿Cómo funciona encolar?: Encolo 6



Size():

Retornar $(N-f+r) \bmod N$

Nota: La cola puede almacenar N-1 elementos a lo sumo

Nota: r apunta a 8 que es el siguiente slot vacío.

enqueue(e):

Si $size() == N-1$ entonces
error("cola llena")

Sino

$q[r] = e$

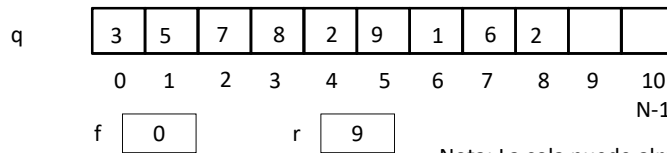
$r = (r+1) \bmod N$

Estructuras de datos - Dr. Sergio A. Gómez

70

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
"Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2026.

¿Cómo funciona encolar?: Encolo 2



Size():
Retornar $(N-f+r) \bmod N$

Nota: La cola puede almacenar N-1 elementos a lo sumo

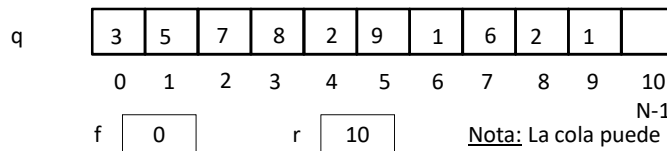
Nota: r apunta a 9 que es el siguiente slot vacío.

enqueue(e):
Si $\text{size()} == N-1$ entonces
 error("cola llena")
Sino
 $q[r] = e$
 $r = (r+1) \bmod N$

Estructuras de datos - Dr. Sergio A. Gómez

71

¿Cómo funciona encolar?: Encolo otro 1



Size():
Retornar $(N-f+r) \bmod N$

Nota: La cola puede almacenar N-1 elementos a lo sumo

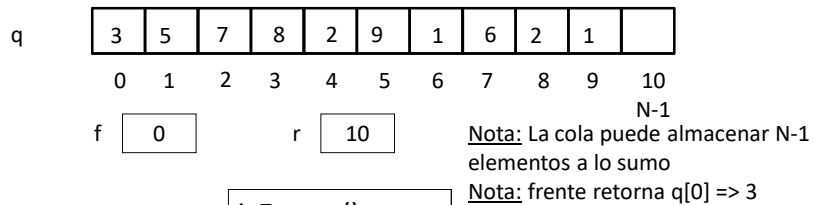
Nota: r apunta a 10 que es el siguiente slot vacío. La cola está llena porque tiene N - 1 elementos

enqueue(e):
Si $\text{size()} == N-1$ entonces
 error("cola llena")
Sino
 $q[r] = e$
 $r = (r+1) \bmod N$

Estructuras de datos - Dr. Sergio A. Gómez

72

¿Cómo funcionan las operaciones frente y desencolar?



isEmpty():
Retornar $f == r$

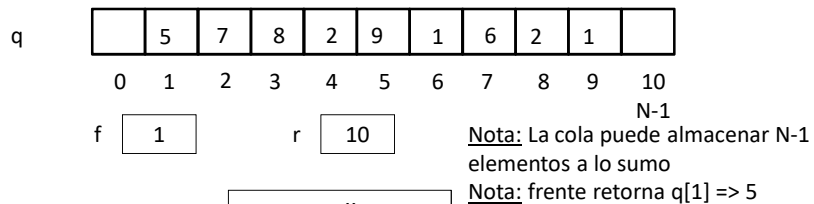
dequeue():
Si isEmpty() entonces error(“cola vacía”)
Sino
temp = q[f]; q[f] = null;
f = (f+1) mod N; retornar temp

front():
Si isEmpty() entonces
error(“cola vacía”)
Sino
retornar q[f]

Estructuras de datos – Dr. Sergio A. Gómez

73

Desencolo 1 vez: Descarté el 3 del frente



isEmpty():
Retornar $f == r$

dequeue():
Si isEmpty() entonces error(“cola vacía”)
Sino
temp = q[f]; q[f] = null;
f = (f+1) mod N; retornar temp

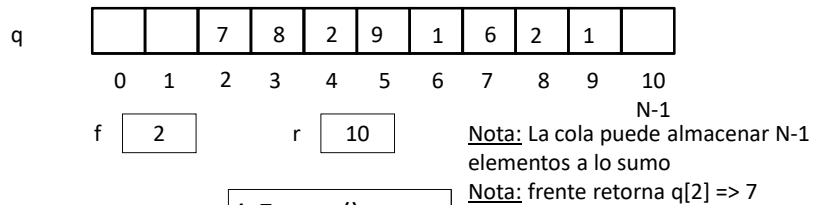
front():
Si isEmpty() entonces
error(“cola vacía”)
Sino
retornar q[f]

Estructuras de datos – Dr. Sergio A. Gómez

74

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2026.

Desencolo otra vez: Descarté el 5 del frente



isEmpty():
 Retornar $f == r$

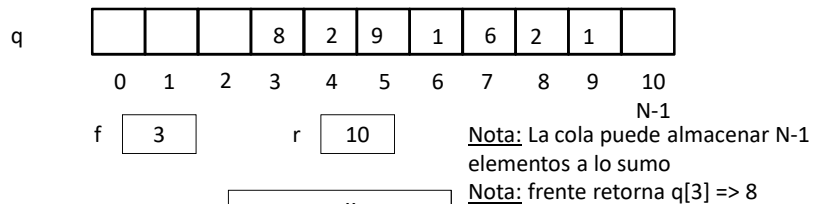
dequeue():
 Si isEmpty() entonces error(“cola vacía”)
 Sino
 temp = q[f]; q[f] = null;
 f = (f+1) mod N; retornar temp

front():
 Si isEmpty() entonces
 error(“cola vacía”)
 Sino
 retornar q[f]

Estructuras de datos – Dr. Sergio A. Gómez

75

Desencolo otra vez más: Descarté el 7 del frente



isEmpty():
 Retornar $f == r$

dequeue():
 Si isEmpty() entonces error(“cola vacía”)
 Sino
 temp = q[f]; q[f] = null;
 f = (f+1) mod N; retornar temp

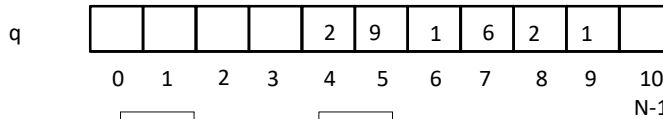
front():
 Si isEmpty() entonces
 error(“cola vacía”)
 Sino
 retornar q[f]

Estructuras de datos – Dr. Sergio A. Gómez

76

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2026.

Desencolo otra vez más: Descarté el 8 del frente



f 4 r 10

Nota: La cola puede almacenar N-1 elementos a lo sumo

Nota: frente retorna q[4] => 2

isEmpty():
Retornar f == r

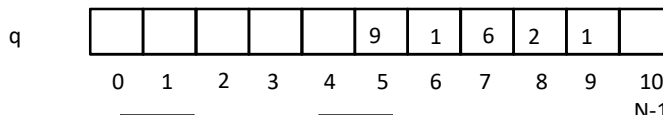
dequeue():
Si isEmpty() entonces error("cola vacía")
Sino
temp = q[f]; q[f] = null;
f = (f+1) mod N; retornar temp

front():
Si isEmpty() entonces
error("cola vacía")
Sino
retornar q[f]

Estructuras de datos – Dr. Sergio A. Gómez

77

Desencolo otra vez más: Descarto el 2 del frente



f 5 r 10

Nota: La cola puede almacenar N-1 elementos a lo sumo

Nota: frente retorna q[5] => 9

isEmpty():
Retornar f == r

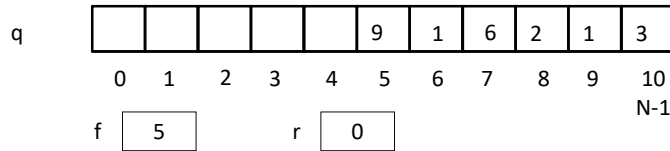
dequeue():
Si isEmpty() entonces error("cola vacía")
Sino
temp = q[f]; q[f] = null;
f = (f+1) mod N; retornar temp

front():
Si isEmpty() entonces
error("cola vacía")
Sino
retornar q[f]

Estructuras de datos – Dr. Sergio A. Gómez

78

Ahora encolamos nuevamente: encolo un 3



Size():
Retornar $(N-f+r) \bmod N$

Nota: La cola puede almacenar N-1 elementos a lo sumo

enqueue(e):
Si $\text{size()} == N-1$ entonces
 error(“cola llena”)
Sino
 $q[r] = e$
 $r = (r+1) \bmod N$

Nota: El 3 va en la posición 10.

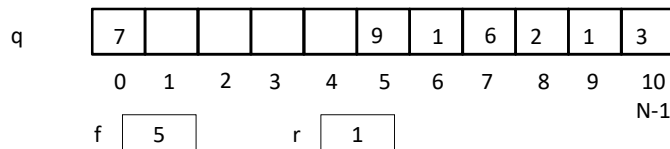
¡Ahora r vale 0!

Ahí está la circularidad del arreglo

Estructuras de datos - Dr. Sergio A. Gómez

79

Ahora encolamos nuevamente: encolo un 7



Size():
Retornar $(N-f+r) \bmod N$

Nota: La cola puede almacenar N-1 elementos a lo sumo

enqueue(e):
Si $\text{size()} == N-1$ entonces
 error(“cola llena”)
Sino
 $q[r] = e$
 $r = (r+1) \bmod N$

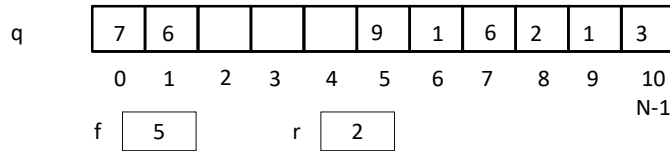
Nota: El 7 va en la posición 0.
¡Ahora r vale 1!

Estructuras de datos - Dr. Sergio A. Gómez

80

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
“Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2026.

Ahora encolamos nuevamente: encolo un 6



Size():
Retornar $(N-f+r) \bmod N$

enqueue(e):
Si $size() == N-1$ entonces
 error(“cola llena”)
Sino
 $q[r] = e$
 $r = (r+1) \bmod N$

Nota: La cola puede almacenar N-1 elementos a lo sumo

Nota: El 6 va en la posición 1.
¡Ahora r vale 2!

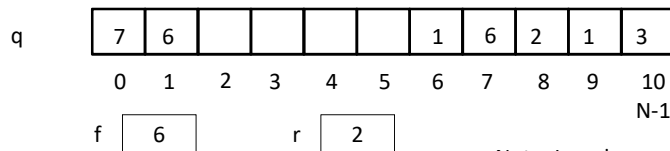
Nota: Entonces hay un agujero en el medio del arreglo que va a ir girando a medida que encolo y desencolo

Nota: $size() = (11-5+2) \% 11 = 8\%11 = 8$

Estructuras de datos - Dr. Sergio A. Gómez

81

Ahora desencolé: tiré el 9 del frente



isEmpty():
Retornar $f == r$

dequeue():
Si $isEmpty()$ entonces error(“cola vacía”)
Sino
 temp = $q[f]$; $q[f] = \text{null}$;
 $f = (f+1) \bmod N$; retornar temp

Nota: La cola puede almacenar N-1 elementos a lo sumo

Nota: f ahora vale 6 y el frente es 1.

Nota: El agujero en el medio del arreglo que va a ir girando a medida que encolo y desencolo

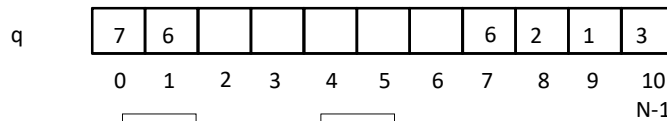
Nota: $size() = (N-f+r) \% N$
 $= (11-6+2) \% 11 = 7$

Estructuras de datos - Dr. Sergio A. Gómez

82

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
“Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2026.

Ahora desencolá de nuevo: tiré el 1 del frente



f 7 r 2

Size():
Retornar $(N-f+r) \bmod N$

dequeue():
Si isEmpty() entonces error(“cola vacía”)
Sino
 temp = q[f]; q[f] = null;
 f = (f+1) mod N; retornar temp

Nota: La cola puede almacenar N-1 elementos a lo sumo

Nota: f ahora vale 7 y el frente es 6.

Nota: Entonces hay un agujero en el medio del arreglo que va a ir girando a medida que encolo y desencolo

Nota: size() = $(11-7+2) \% 11$
= $6 \% 11 = 6$

Estructuras de datos – Dr. Sergio A. Gómez

83

Repositorio de archivos

Usaremos un repositorio de Github donde están los archivos de las interfaces y los casos de testeo en formato Junit.

<https://github.com/ed-dcic-uns/tdas>

```

1 package ar.edu.uns.cs.ed.tdas.tdapila;
2
3 /**
4  * Interface Stack
5  * @author Cátedra de Estructuras de Datos, Departamento de Cs. e Ing. de la Computación, UNS.
6  */
7 public interface StackEx {
8
9     /**
10      * Consulta la cantidad de elementos de la pila.
11      * @return Cantidad de elementos de la pila.
12      */
13     public int size();

```

84

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
“Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2026.

Bibliografía

- Goodrich & Tamassia, Data Structures and Algorithms in Java, 4th edition, John Wiley & Sons, 2006, Capítulo 5

85